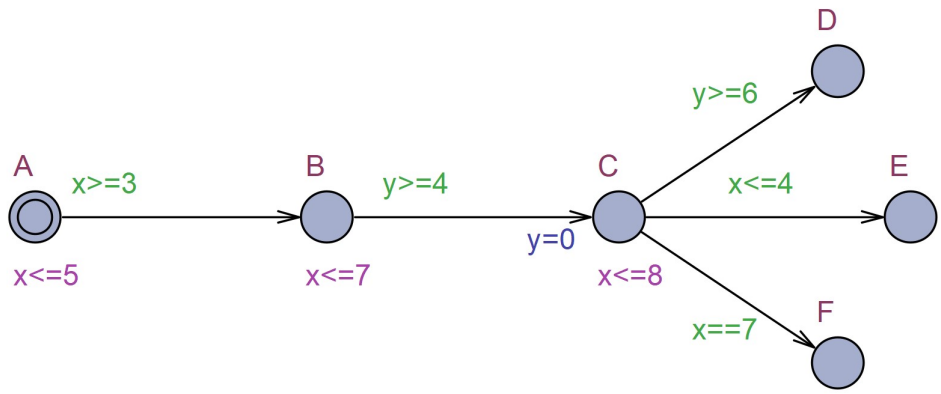
Suman Sourav
Jonathan J. Huerta y Munive

AALBORG UNIVERSITY
DENMARK

Models and Tools for Cyber-Physical Systems
Exercise sheet 7
WITH SOLUTIONS

Exercise 1: Symbolic Exploration of Timed Automata

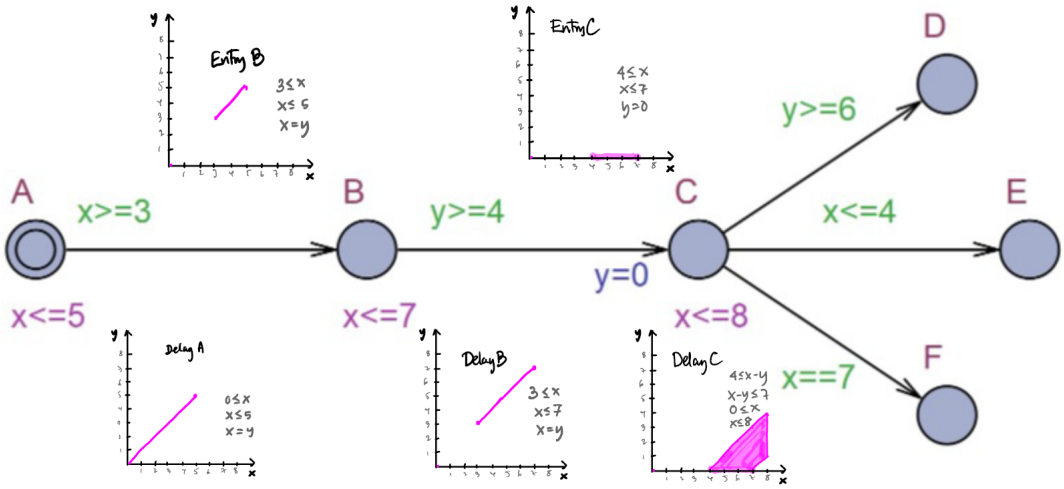
Consider the timed automaton $\mathcal{A}$ below with two clocks `x` and `y`. Perform a symbolic zone-base reachability analysis of $\mathcal{A}$ as in Lecture 6 (slides 37-44). Which of the locations `D`, `E` and `F` are reachable? Describe, using difference constraints, the reachable zones upon entry and after delay for the locations `A`, `B` and `C`.



You can find a UPPAAL model of $\mathcal{A}$ in SimpleTA.xml in Moodle.

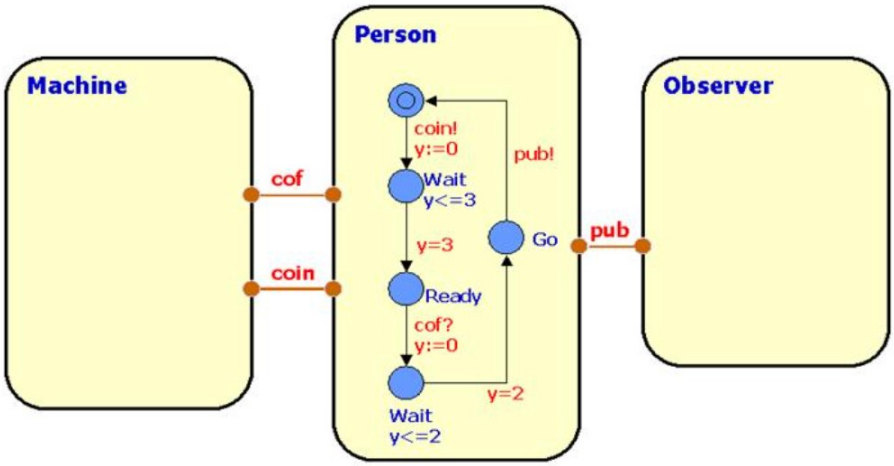
. . . . . Solution sketch . . . . . .

Locations `E` and `F` are reachable, whereas locations `D` is not reachable. The symbolic zone-based reachability of $\mathcal{A}$ is given below:



Exercise 2: Coffee Machine

In this exercise you are asked to design the control of a `Machine` (the control program) which will serve a coffee craving `Person` (the environment). As you can see below the person repeatedly (tries to) insert a `coin`, (tries to) extract `cof`fee after which (s)he will make a `pub`lication. Between each action the `Person` requires a suitable time-delay before being ready to participate in the next one.



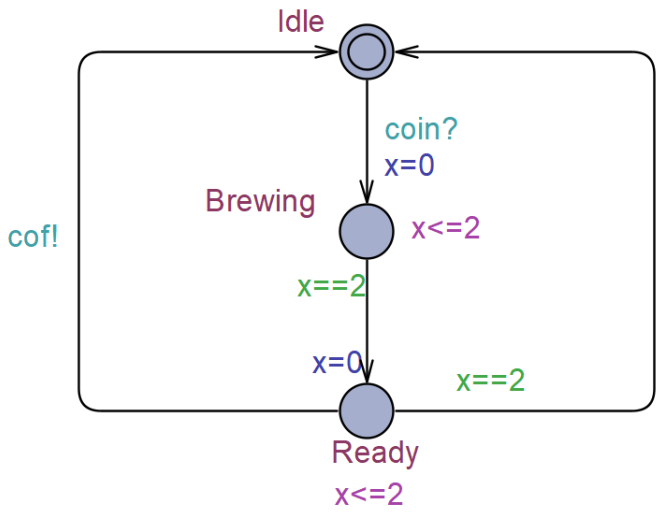
The `Machine` takes some time for brewing the coffee and will time-out if coffee has not been taken before a certain upper time-limit.

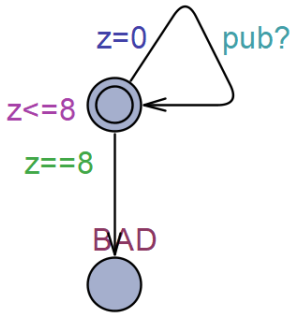
As a requirement we want the overall behaviour to ensure that the indicated `Observer` experiences a constant flow of publications from the system. In particular we want the `Observer` to complain if at any time more than 8 time-units elapses between two consecutive publications. Model the `Machine` and `Observer` in UPPAAL and analyse the behaviour of the system. Try to determine the maximum brewing time allowed by the `Machine` in order that the above requirement is met.

An initial sketch of a UPPAAL can be found in CoffeeMachine.xml in Moodle.

. . . . . Solution sketch . . . . . .

Below you find a proposal for `Machine` and `Observer`. In the suggested solution the brewing time of the `Machine` is 2 time-units.





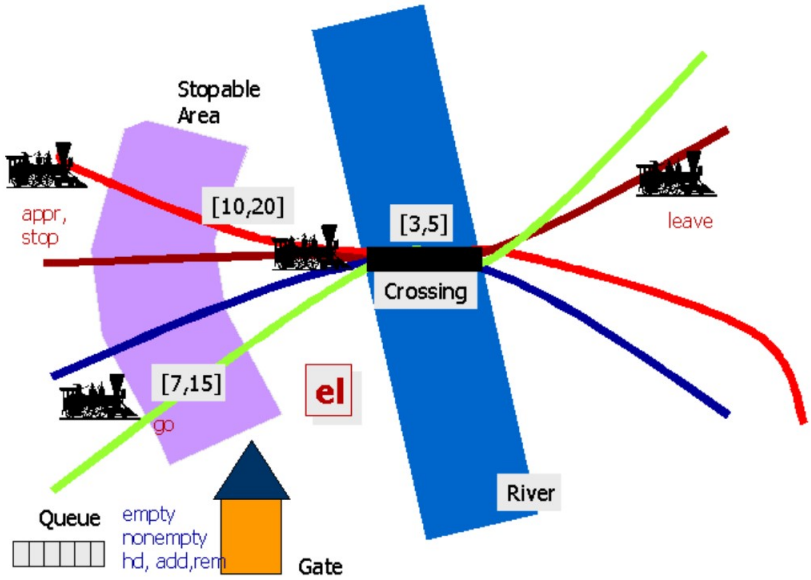
To verify that the `Observer` does not complain we check the following UPPAAL query

```
A[] !  Observer.BAD
```

Now re-verifying the above query with changing brewing times of the `Machine` we find that the maximum brewing where the `Observer` does not complain is 5 time-units.
You may find a complete solution in CoffeeMachineSol.xml on Moodle.

Exercise 3: Train-Gate Error Correction

Below is the overview of the `TrainGate` model. An erroneous UPPAAL model of this case may be found in train-gate-err.xml on Moodle and in train-gate-err.q you will find the properties.



Please do the following.

(a) Use UPPAAL (simulation, verification, or whatever you want) to pin-point and explain the error(s).

(b) Having corrected the errors, what is the minimum time one can guarantee between a train requesting access to the crossing and actually getting there?

(c) Having corrected the errors, what is the minimum time one can guarantee between a train being stopped and getting the permission to start again (i.e., the waiting time)?

.....Solution sketch .....

(a) There are two errors both in the `Gate` component. 1) In the bottom-most location time can elapse arbitrarily – that is there is no time-bound between a train signals that it is approaching and it will be stopped. You should make the location committed. 2) The function for returning the front element has an index-error. The front element is `list[0]` and not `list[1]`.

(b) The minimum time is 80. You can find this by inserting a new clock `z` to the `Train` template and verifying queries of the type `Train(0).Appr --> (Train(0).Cross && Train(0).z<=B)` with different values of `B`. You can use binary search to find the smallest value of `B` that will make the query true.

(c) The minimum time is 65. You can use the same method as for the previous question, but now the transition from (Appr) to (Stop) should reset `z` to zero. Then you can verify queries of the type `Train(0).Stop --> (Train(0).Start && Train(0).z<=B)` with different values of `B`.